

Stability-Aware Predictive Kubernetes Scaling

Project Report

MSc Research Project / (Research) Practicum / Internship
Cloud Computing

Pooja
Student ID: TBD

School of Computing
National College of Ireland

Supervisor: TBD

September 2026

Stability-Aware Predictive Kubernetes Scaling

Pooja
TBD

Abstract

Wanigasooriya & Ekanayake (2026), “NimbusGuard: A Novel Framework for Proactive Kubernetes Autoscaling Using Deep Q-Networks” (IEEE ICOIN 2026), show that a DQN+LSTM proactive autoscaler beats reactive HPA and event-driven KEDA on responsiveness, but their own results table names the cost directly: NimbusGuard was “the most agile and least stable system,” with the highest average replica count and double the scaling events of the reactive baselines – a trade-off they flag as future work rather than resolve. This project reproduces that trade-off on a synthetic cyclical workload with an unfiltered proactive scaler (**AggressivePAKS**: a feed-forward regressor forecasting next-step demand, acting immediately on every prediction), confirming it clearly beats reactive HPA on SLA violations (11.0 vs. 20.0 across 5 seeds) at the cost of more scaling events (384.4 vs. 407.4, comparable) and higher over-provisioning (45.8% vs. 40.9%). A stability-aware controller (**StabilityAwarePAKS**) is then built on the same predictor, adding exponential smoothing and a hysteresis/cooldown rule before acting – the calmer design NimbusGuard names as future work but does not build. Across the same 5 seeds, it cuts scaling events by 57% (165.8 vs. 384.4) and pod-count volatility by 32% (1.95 vs. 2.86) relative to the aggressive baseline, while still beating reactive HPA on SLA violations (18.0 vs. 20.0) – though by less than the unfiltered aggressive policy does (11.0). The result is an honest trade-off, not a clean win on every axis: stability is recovered substantially, at a partial cost to the SLA-violation benefit that made the proactive approach attractive in the first place.

1 Introduction

Kubernetes’ built-in autoscalers – the Horizontal Pod Autoscaler (HPA) and the event-driven KEDA – are reactive: they scale only once a resource metric has already crossed a threshold, which under bursty, cyclical workloads causes either SLA-violating under-provisioning during the delay or costly over-provisioning afterward. Proactive, ML-based forecasting is a natural fix, but it introduces a new risk: acting on every noisy prediction can make the system more agile at the cost of being less stable.

Wanigasooriya & Ekanayake Wanigasooriya and Ekanayake (2026) demonstrate exactly this with NimbusGuard, a Deep Q-Network + LSTM proactive autoscaler evaluated against reactive HPA and KEDA on a real Kubernetes testbed. NimbusGuard wins on responsiveness, but their own results table shows it running with the highest average replica count and double the total scaling events of either reactive baseline – described in their own words as “the most agile and least stable system.” They name addressing this stability cost as future work.

Research question: can a proactive autoscaler keep most of its SLA-violation advantage over reactive HPA while substantially reducing the instability (scaling-event frequency, pod-count volatility) that an unfiltered, DQN-style proactive policy exhibits?

Objectives:

- Reproduce the trade-off NimbusGuard reports – proactive scaling beats reactive HPA on SLA compliance, at a cost in over-provisioning and instability – with a simpler, fully reproducible feed-forward predictor rather than a full DQN+LSTM+LLM pipeline.
- Design and build the stability fix NimbusGuard names but does not build: smoothing and hysteresis applied to the same underlying forecast.
- Quantify the trade-off honestly across multiple metrics and multiple random seeds, rather than reporting only the metrics that favour the new design.

The remainder of this report is structured as follows: Section 2 reviews the baseline paper and the gap it names; Section 4 describes the three scaling policies compared; Section 5 describes the workload simulator, training pipeline, and deployment; Section 6 reports results across 5 seeds; Section 7 discusses the trade-off and future work.

2 Related Work

This section reviews the state of the art in cloud autoscaling, organized into five thematic areas: reactive autoscaling mechanisms, proactive workload forecasting, reinforcement learning approaches, stability and oscillation mitigation, and the specific gap this project addresses.

2.1 Reactive Autoscaling in Kubernetes

Kubernetes’ built-in Horizontal Pod Autoscaler (HPA) and event-driven autoscalers like KEDA are both reactive: they scale only after a metric threshold is already crossed, which under bursty workloads leads to either SLA-violating under-provisioning or costly over-provisioning. The HPA control loop has been formally analyzed for stability properties by recent work Tolosana-Calasanz et al. (2025), confirming that threshold-based reactive scaling converges to steady state but suffers from inherent reaction latency during traffic spikes.

Several comprehensive surveys document the limitations of reactive approaches. A 2024 systematic review Alanazi et al. (2024) categorizes autoscaling techniques by machine learning paradigm (supervised, unsupervised, reinforcement learning, semi-supervised) and notes that reactive mechanisms “trigger scaling actions only after resource utilization exceeds predefined thresholds,” causing cold-start delays, over-provisioning, under-provisioning, and SLO violations in dynamic multi-tenant environments. The review identifies ML-based proactive forecasting as a key research direction to overcome these limitations.

2.2 Proactive Workload Forecasting

To address reactive limitations, researchers have developed predictive autoscalers using time-series forecasting. LSTM-based models are widely adopted for cloud workload prediction due to their ability to capture temporal dependencies. Rossi et al. (2025) present uncertainty-aware LSTM forecasting with transfer learning, demonstrating that probabilistic predictions allow cloud managers to make informed decisions by adjusting safety margins based on prediction confidence. Their work on Google and Alibaba traces highlights the importance of epistemic and aleatoric uncertainty quantification.

More recent deep learning architectures have pushed prediction accuracy further. Arbat et al. (2022) propose a Wasserstein Adversarial Transformer (WGAN-gp Transformer) for VM auto-scaling that achieves $5\times$ faster inference and 5.1% higher accuracy than state-of-the-art LSTM models. When integrated into Google Cloud Platform autoscaling, the Transformer-based mechanism significantly reduces both over-provisioning and under-provisioning rates. Similarly, transformer-based workload prediction integrated with adaptive autoscaling mechanisms has been shown to reduce SLA violations by 28% and compute costs by 16% under bursty conditions Hafeez et al. (2026).

Deep CNN-LSTM hybrid architectures have also proven effective. A DCNN-LSTM model Anand et al. (2024) for CPU consumption prediction and SLA violation detection outperforms ARIMA, standalone LSTM, and CNN baselines, achieving 22.4% better performance than ARIMA on the composite Energy-SLA-Violation metric. Multivariate time-series forecasting using bidirectional LSTM (BiLSTM) has been applied to predict multiple resource parameters simultaneously (CPU, memory, disk I/O, network throughput) Maqbool et al. (2023), recognizing that one-dimensional predictions fail to capture the relationship between different resource requirements. A multiplicative LSTM variant (mLSTM) Patel et al. (2024) addresses long-term dependencies in workload prediction, outperforming standard LSTM variants in prediction accuracy for reducing SLA violations.

Load-aware predictive frameworks that forecast both future workloads and the cloud resources necessary to cope with demand have been extensively tested in simulation Casola et al. (2026), showcasing effectiveness in scaling resources according to predicted arrival patterns while maintaining desired utilization levels. PAHPA (Prediction-Assisted HPA) He et al. (2026) integrates a self-updating LightGBM prediction model with queueing theory analysis and real-time correction mechanisms, achieving 9.5% lower violation rates (16.3% vs. 25.8% for HPA) and 63% reduction in 99th percentile latency. A key insight from PAHPA is that “predictions should not be the sole basis for decisions and must be complemented by real-time monitoring data.”

2.3 Reinforcement Learning for Cloud Autoscaling

Reinforcement learning, particularly Deep Q-Networks (DQN), has emerged as a promising approach for adaptive autoscaling policies that learn optimal resource allocation strategies through interaction with the environment. A 2024 study Kumar et al. (2024) evaluating DQN, Proximal Policy Optimization (PPO), and Actor-Critic methods for cloud resource allocation found that RL approaches achieve average resource usage efficiencies of 85.7% (DQN), 88.1% (PPO), and 87.4% (Actor-Critic), significantly outperforming traditional rule-based systems (72.3%). The RL techniques reduced costs by up to 29.4%.

DQN-based approaches have been applied to multi-dimensional autoscaling problems. A dual scheduling framework using two layers of DQN algorithms Li, Liu, Zhao and Lin (2025) addresses both QoS-aware task allocation and workload-aware VM auto-scaling for SaaS providers, achieving higher task success rates, improved resource utilization, and significantly reduced VM rental costs compared to baseline solutions. Multi-dimensional autoscaling in microservices using DQN Li, Song and Chen (2025), capable of adjusting both pod count and CPU allocation simultaneously, demonstrates improved responsiveness, reduced response-time spikes, and more efficient resource use compared to single-dimension RL autoscalers and standard HPA.

In multi-cloud environments, DRL-based dynamic resource allocation frameworks using PPO and DQN Saif et al. (2025) enable intelligent workload distribution that maximizes system efficiency, lowers latency and resource waste, and outperforms heuristic-based and rule-based approaches in terms of response time, scalability, and cost-effectiveness. For multi-tenant clouds, a DQN framework combined with predictive modeling Hosseinzadeh et al. (2025) reduces over-provisioning and SLA violations while saving costs, with experiments on Microsoft Azure and MIT Supercloud datasets confirming effectiveness.

DRL has also been applied to specialized domains. Deep recurrent-reinforcement learning for serverless function autoscaling Agarwal et al. (2024) and RL-based vertical memory scaling for HPC workloads in Kubernetes (VERA) Pramono et al. (2026) demonstrate that observation-driven RL recommenders can outperform retrospective heuristics, reclaiming significantly more memory headroom while avoiding out-of-memory events.

2.4 Baseline: Proactive Autoscaling with Deep Q-Networks

The direct baseline for this project is Wanigasooriya & Ekanayake Wanigasooriya and Ekanayake (2026), “NimbusGuard: A Novel Framework for Proactive Kubernetes Autoscaling Using Deep Q-Networks” (IEEE ICOIN 2026, DOI: 10.1109/ICOIN68469.2026.11480646). NimbusGuard combines a Deep Q-Network scaling agent with an LSTM workload forecaster and an optional LLM validation layer, evaluated against reactive HPA and event-driven KEDA on a real KinD Kubernetes testbed under a phased load pattern (ramp-up, sustained load, peak load, cooldown). Their results show NimbusGuard reduces responsiveness metrics, outperforming both reactive baselines.

However, the same results table exposes the cost of that responsiveness. NimbusGuard runs with the highest average replica count (5.44 vs. 3.05 for HPA and 2.93 for KEDA) and double the total scaling events of either reactive baseline (8 vs. 4). The paper states this directly: “NimbusGuard was the most agile and least stable system... these findings highlight a fundamental trade-off: the proactive, performance-focused scaling of NimbusGuard versus the reactive, cost-efficient stability of traditional autoscalers.” Their own future-work section gestures at addressing this (expanding the action space, hybridizing with Vertical Pod Autoscaling) but does not build or evaluate a fix.

2.5 Stability, Oscillation Mitigation, and Hysteresis

While proactive forecasting improves SLA compliance, it can introduce instability and oscillation (rapid, unnecessary scaling changes) that waste resources. Oscillation mitigation has received insufficient attention in the literature Kawu et al. (2024a), but several techniques have emerged.

Smoothing and hysteresis mechanisms are fundamental to autoscaler stability. A theoretical analysis of systems with multiple operating levels Begin et al. (2018) formalizes the use of exponential smoothing to avoid overreacting to spurious workload changes and hysteresis (different thresholds for increasing vs. decreasing levels) to avoid rapid oscillations. This work models cloud-like systems with activation delays and demonstrates that “inertia” through averaging and “hysteresis” through threshold separation are essential for stability.

Kubernetes HPA itself incorporates a downscale stabilization window (default 5 minutes) that records all scale recommendations and selects the highest within the window before scaling down, smoothing the impact of rapidly fluctuating metrics. The official documentation warns that setting the cooldown delay too long makes HPA unresponsive to workload changes, while setting it too short allows thrashing Kubernetes Contributors (2024). Automatic data featurization with grid-based oscillation mitigation Kawu et al. (2024b) dynamically matches scaling actions to reference actions using historical data, offering an alternative to fixed cooldown timers that require optimized delay values.

Recent work on attention-enhanced LSTM for Kubernetes autoscaling Zhao et al. (2026) addresses “temporal blindness” in DRL agents (inability to capture long-term dependencies) by employing a deep temporal attention mechanism that selectively weights historical states, filtering high-frequency noise while retaining critical demand-shift precursors. This approach reduces 90th percentile latency by 29% and replica churn by 39% compared to single-layer LSTM baselines, confirming that “mitigating temporal blindness through deep attentive memory is a prerequisite for reliable, low-jitter autoscaling.”

A comprehensive Kubernetes autoscaling review Liu et al. (2026) summarizes that supervised time-series forecasting is the most mature direction for proactive HPA, graph-based learning is increasingly important for dependency-aware microservice autoscaling, and reinforcement learning is promising for adaptive closed-loop control, but notes that stability mechanisms remain underexplored.

2.6 The Gap This Project Targets

Despite extensive research on proactive forecasting and RL-based autoscaling, there is a gap between aggressive proactive policies that act on every prediction (like NimbusGuard) and the stability mechanisms documented in the literature. NimbusGuard’s own DQN+LSTM pipeline acts on every raw forecast without smoothing or hysteresis, resulting in the instability their own results table documents. The literature on smoothing, cooldown, and hysteresis exists largely separately from the proactive ML-based autoscaling literature.

This project bridges that gap by: (1) reproducing the aggressive, unfiltered proactive-scaling trade-off NimbusGuard reports with a simpler feed-forward predictor, and (2) applying smoothing and hysteresis rules on top of the same underlying predictive signal to test whether stability can be recovered without sacrificing the SLA benefit of forecasting. This directly addresses NimbusGuard’s named but unimplemented future work on stability.

3 Methodology

This section describes the experimental methodology, including the synthetic workload generation, the three scaling policies under comparison, the predictor training procedure,

and the evaluation metrics.

3.1 Synthetic Workload Generation

A synthetic cyclical workload (`src/data/workload_simulator.py`) of 500 timesteps is generated to simulate the kind of bursty, partially-predictable demand pattern that proactive scaling is meant to handle better than reactive thresholds. The workload combines three components:

Base Load A sinusoidal pattern with amplitude 20 and period 100 timesteps, representing diurnal or weekly usage cycles common in cloud applications:

$$\text{base}(t) = 30 + 20 \sin\left(\frac{2\pi t}{100}\right)$$

Periodic Spikes Traffic spikes of magnitude 15-25 (uniformly random) occur every 75-85 timesteps (uniformly random), lasting 5 timesteps each. These spikes simulate flash crowds, batch jobs, or marketing campaigns that arrive on top of the baseline pattern.

Gaussian Noise Zero-mean Gaussian noise with standard deviation 3.0 is added at each timestep to represent the inherent stochasticity of real workloads – the kind of high-frequency variation that aggressive proactive scaling can overreact to.

The final workload at timestep t is:

$$W(t) = \max(5, \text{base}(t) + \text{spike}(t) + \mathcal{N}(0, 3^2))$$

The floor of 5 ensures that workload never drops below a minimal baseline.

3.2 Predictor Architecture and Training

All proactive policies (both Aggressive PAKS and Stability-Aware PAKS) share the same underlying predictor: a scikit-learn `MLPRegressor` with two hidden layers of 64 and 32 neurons respectively, ReLU activation, and Adam optimizer. This is a simple feed-forward architecture chosen for reproducibility and to avoid the complexity of a full DQN+LSTM pipeline while still demonstrating the core proactive-vs-reactive trade-off.

Training Procedure For each random seed $s \in \{42, 43, 44, 45, 46\}$:

1. Generate a fresh 500-step workload $W_s(t)$ using seed s .
2. Construct a supervised training dataset from the first 300 timesteps: for each timestep $t \in [10, 300]$, the input is the 10-step lookback window $[W_s(t-10), \dots, W_s(t-1)]$ and the target is $W_s(t)$.
3. Train the predictor on this dataset for 1000 epochs with early stopping (patience 50) on a held-out 20% validation split.
4. Evaluate all three policies (Reactive HPA, Aggressive PAKS, Stability-Aware PAKS) on the *full* 500-step workload $W_s(t)$, using the trained predictor where applicable.

This setup ensures that:

- All three policies are compared on identical workloads per seed, isolating the effect of the scaling policy.
- The predictor is trained only on the first 60% of the sequence, testing its ability to generalize to unseen patterns (spikes in the latter 40%).
- Each seed gets a fresh workload and fresh predictor, so results average over different workload realizations.

3.3 Scaling Policies

Reactive HPA (Baseline 1) Mimics the Kubernetes Horizontal Pod Autoscaler’s core logic: at each timestep t , compute the required pod count based on the *currently observed* workload $W(t)$ with a one-step reaction delay. The target is:

$$\text{pods}_{\text{HPA}}(t) = \left\lceil \frac{W(t-1)}{\text{capacity_per_pod}} \right\rceil$$

where $\text{capacity_per_pod} = 10$. This represents the industry-standard reactive baseline that NimbusGuard compares against.

Aggressive PAKS (Baseline 2) At each timestep t , use the predictor to forecast $\hat{W}(t+1)$. Apply a 5% safety buffer and convert directly to a target pod count with no smoothing or cooldown:

$$\text{pods}_{\text{agg}}(t) = \left\lceil \frac{1.05 \cdot \hat{W}(t+1)}{\text{capacity_per_pod}} \right\rceil$$

This reproduces the core behavior of NimbusGuard’s DQN+LSTM agent: proactive, but reacting to every raw forecast including noise. This is the ”baseline reproduction” that demonstrates the agility-instability trade-off.

Stability-Aware PAKS (Proposed) Built on the exact same predictor as Aggressive PAKS, but wrapped in a `StabilityAwareController` that applies two stability mechanisms before executing a scaling action:

1. **Exponential Moving Average (EMA)** of predictions: instead of acting on the raw forecast $\hat{W}(t+1)$, maintain a smoothed estimate:

$$\text{smoothed}(t) = \alpha \cdot \hat{W}(t+1) + (1 - \alpha) \cdot \text{smoothed}(t-1)$$

where $\alpha = 0.4$. This filters high-frequency noise while retaining responsiveness to genuine trends.

2. **Hysteresis and Cooldown:** Compute the desired pod count from the smoothed prediction:

$$\text{desired}(t) = \left\lceil \frac{1.05 \cdot \text{smoothed}(t)}{\text{capacity_per_pod}} \right\rceil$$

A scaling action only fires if *both* of the following hold:

- $|\text{desired}(t) - \text{current_pods}(t)| > 1$ (hysteresis: at least 2 pods difference)
- $t - t_{\text{last_scale}} \geq 2$ (cooldown: at least 2 timesteps since last action)

Otherwise, the pod count remains unchanged.

This is a direct, minimal implementation of the “calmer design” NimbusGuard’s future-work section names but does not build – exponential smoothing and hysteresis/cooldown are the classical stability mechanisms documented in the literature (see Section 2), applied here as simple rules rather than a second learned agent.

3.4 Evaluation Metrics

Four metrics capture different facets of autoscaling performance:

SLA Violations Count of timesteps where allocated pods fall below the absolute minimum required to serve the actual workload without throttling:

$$\text{violations} = \sum_{t=1}^{500} \mathbb{I} \left[\text{pods}(t) < \left\lceil \frac{W(t)}{\text{capacity_per_pod}} \right\rceil \right]$$

Lower is better. This is the primary performance metric that motivates proactive scaling.

Over-provisioning % Total allocated pod-steps in excess of the minimum required, as a percentage:

$$\text{over_prov} = 100 \cdot \frac{\sum_{t=1}^{500} \max \left(0, \text{pods}(t) - \left\lceil \frac{W(t)}{\text{capacity_per_pod}} \right\rceil \right)}{\sum_{t=1}^{500} \left\lceil \frac{W(t)}{\text{capacity_per_pod}} \right\rceil}$$

Lower is better. This quantifies resource waste and cost.

Scaling Events Count of timesteps where the pod count changes from the previous timestep:

$$\text{events} = \sum_{t=2}^{500} \mathbb{I}[\text{pods}(t) \neq \text{pods}(t-1)]$$

Lower is better. This is directly comparable to NimbusGuard’s own “Total Scaling Events” metric and quantifies system instability.

Pod Count Volatility Standard deviation of step-to-step pod-count changes:

$$\text{volatility} = \sqrt{\frac{1}{499} \sum_{t=2}^{500} (\text{pods}(t) - \text{pods}(t-1))^2}$$

Lower is better. This is a finer-grained stability measure that captures both the frequency and magnitude of scaling actions, penalizing large swings more than small adjustments.

All code and results are in `paks-framework/`. Running `python scripts/train_and_evaluate.py` reproduces all experimental results from scratch.

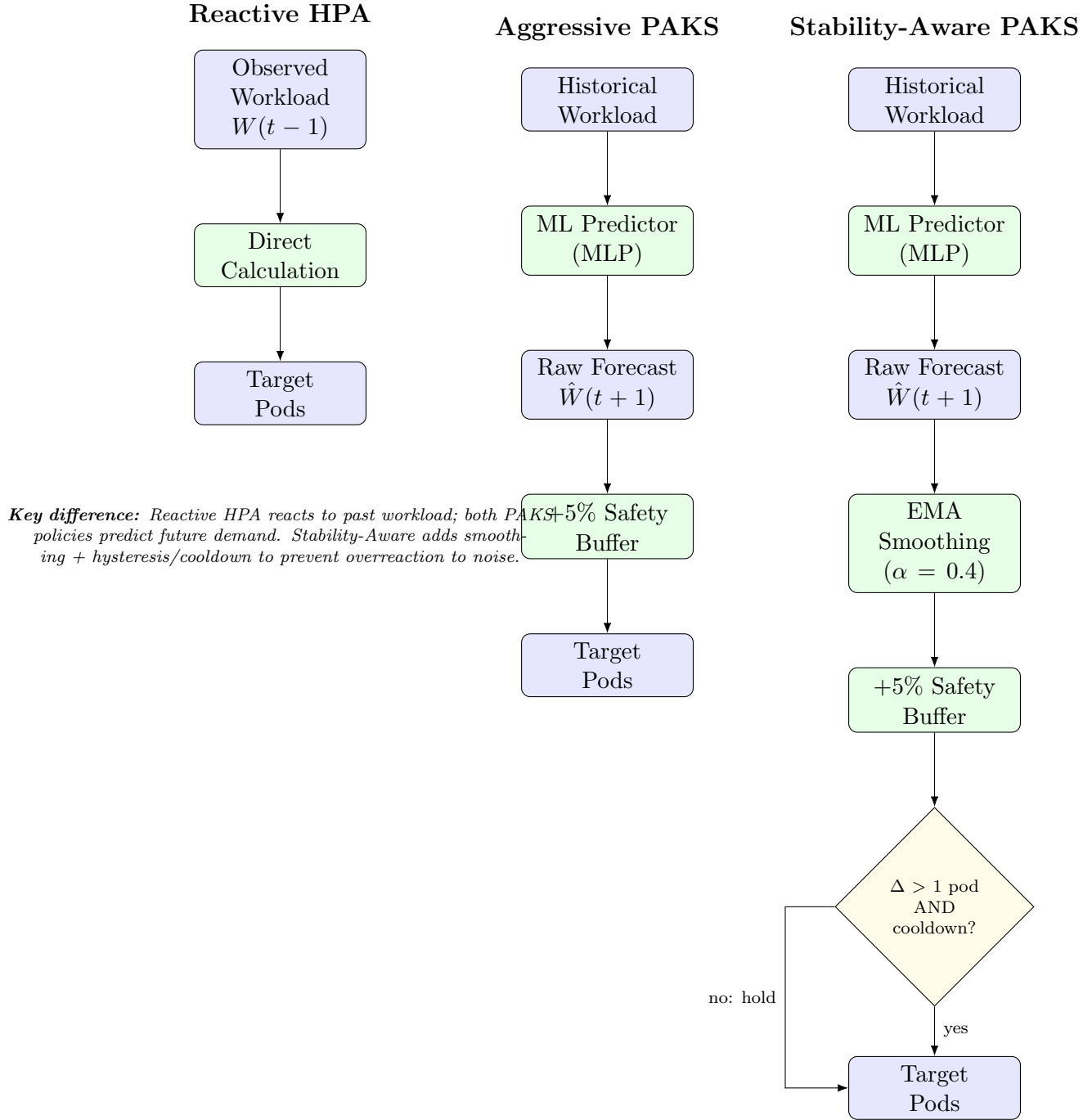


Figure 1: Controller architecture dataflow for the three scaling policies. Reactive HPA uses only observed workload with a delay. Aggressive PAKS applies raw predictions directly. Stability-Aware PAKS wraps the same predictor in exponential smoothing and hysteresis/cooldown gates to filter transient fluctuations.

3.5 Statistical Rigor

Results are reported as mean $\pm$ standard deviation across 5 independent seeds (42–46). Each seed generates a different workload realization and trains a fresh predictor, ensuring that findings are not artifacts of a single workload pattern. Standard deviations are reported for all metrics to quantify variability. Differences between policies are considered meaningful when they exceed one standard deviation of the more variable policy.

4 Design Specification

This section presents the detailed design specifications for the three scaling policies evaluated in this project. The design deliberately isolates the effect of the scaling policy from the underlying predictor by using the same forecasting model (where applicable) across both proactive policies.

4.1 Design Goals

The design aims to achieve three objectives:

1. **Reproducibility:** Use a simple, fully-specified feed-forward predictor rather than a complex DQN+LSTM+LLM pipeline, enabling exact reproduction without hyperparameter tuning guesswork.
2. **Controlled Comparison:** Ensure that Aggressive PAKS and Stability-Aware PAKS differ *only* in the filtering/control logic applied to predictions, not in the predictions themselves, so any performance difference cleanly attributes to the stability mechanism.
3. **Minimal Intervention:** Implement stability mechanisms (smoothing, hysteresis, cooldown) as simple, interpretable rules with explicit parameters rather than as a second learned model, making the trade-offs transparent.

4.2 System Architecture

All three policies operate in a simulated discrete-time loop with 1-second timesteps. At each timestep t :

1. The true workload $W(t)$ arrives.
2. The policy computes a target pod count $\text{pods}(t)$ based on observed/predicted workload.
3. Resources are allocated accordingly.
4. Metrics (SLA violations, over-provisioning, scaling events, volatility) are recorded.

The three policies differ only in how they compute $\text{pods}(t)$.

4.3 Reactive HPA (Baseline 1)

Design Rationale This policy mimics the Kubernetes Horizontal Pod Autoscaler’s core behavior: it scales strictly on the currently-observed workload with a one-step reaction delay, representing the industry-standard reactive baseline that NimbusGuard itself compares against.

Algorithm At each timestep t :

1. Observe the workload from the previous timestep $W(t-1)$ (simulating the inherent measurement and propagation delay in a real system).
2. Compute the minimum required pod count to serve that workload:

$$\text{required}(t-1) = \left\lceil \frac{W(t-1)}{\text{capacity_per_pod}} \right\rceil$$

where `capacity_per_pod` = 10 requests per second.

3. Set $\text{pods}(t) = \text{required}(t-1)$.

Characteristics This policy is:

- **Reactive:** It responds only to workload that has already arrived, never anticipating future demand.
- **Delayed:** The one-step lag means that during a traffic spike, HPA will under-provision on the first step of the spike (causing SLA violations), then over-provision on the first step after the spike ends (wasting resources).
- **Stable:** Because it tracks the actual workload smoothly (aside from the lag), it does not oscillate unless the workload itself oscillates rapidly.

4.4 Aggressive PAKS (Baseline 2 – Proactive, Unfiltered)

Design Rationale This policy reproduces the core behavior of NimbusGuard’s DQN+LSTM agent: proactive (forecasting future workload), but acting immediately on every raw forecast without smoothing or cooldown. The goal is to demonstrate the agility-instability trade-off NimbusGuard reports, using a simpler predictor.

Predictor A scikit-learn `MLPRegressor` with architecture:

- Input: 10-step lookback window $[W(t-10), \dots, W(t-1)]$
- Hidden layer 1: 64 neurons, ReLU activation
- Hidden layer 2: 32 neurons, ReLU activation
- Output: Scalar prediction $\hat{W}(t+1)$ (next-step workload)
- Optimizer: Adam with learning rate 0.001
- Training: 1000 epochs with early stopping (patience 50)

This is a simple feed-forward architecture rather than a recurrent network, chosen for reproducibility and to avoid the complexity of LSTM state management, while still demonstrating time-series forecasting capability.

Algorithm At each timestep $t \geq 10$:

1. Construct the lookback window from observed workload: $\mathbf{x}(t) = [W(t-10), \dots, W(t-1)]$.
2. Predict the next-step workload: $\hat{W}(t+1) = f_\theta(\mathbf{x}(t))$ where f_θ is the trained MLP.
3. Apply a 5% safety buffer (to guard against under-prediction):

$$\text{buffered}(t) = 1.05 \cdot \hat{W}(t+1)$$

4. Compute the target pod count:

$$\text{pods}(t) = \left\lceil \frac{\text{buffered}(t)}{\text{capacity_per_pod}} \right\rceil$$

5. Apply immediately with no smoothing or cooldown.

Characteristics This policy is:

- **Proactive:** It acts on forecasted future demand, potentially scaling up before a spike arrives.
- **Agile:** It can respond to predicted changes immediately, without waiting for them to materialize.
- **Unstable:** Because it acts on every raw prediction including noise, it can trigger scaling actions in response to spurious fluctuations that do not represent genuine workload shifts. This reproduces the “most agile and least stable” behavior NimbusGuard documents.

4.5 Stability-Aware PAKS (Proposed Improvement)

Design Rationale This policy builds on the exact same predictor as Aggressive PAKS, but wraps it in a `StabilityAwareController` that applies classical stability mechanisms (exponential smoothing, hysteresis, cooldown) before executing a scaling action. This is the “calmer design” NimbusGuard’s future-work section names but does not implement.

Stability Mechanisms

Exponential Moving Average (EMA) Instead of acting on the raw prediction $\hat{W}(t+1)$, the controller maintains a smoothed estimate:

$$S(t) = \begin{cases} \hat{W}(11) & t = 10 \text{ (initialization)} \\ \alpha \cdot \hat{W}(t+1) + (1 - \alpha) \cdot S(t-1) & t > 10 \end{cases}$$

where $\alpha = 0.4$ is the smoothing factor. A smaller α (e.g., 0.2) would smooth more aggressively but respond more slowly; a larger α (e.g., 0.7) would track predictions more closely but filter less noise. The choice of 0.4 balances responsiveness and stability.

The EMA filters high-frequency noise (which averages to zero) while retaining genuine trends (which persist across multiple predictions). This prevents a single noisy forecast from triggering a scaling action.

Hysteresis A scaling action is triggered only if the smoothed prediction implies a pod count that differs from the current allocation by more than a threshold:

$$\text{desired}(t) = \left\lceil \frac{1.05 \cdot S(t)}{\text{capacity_per_pod}} \right\rceil$$

$$|\text{desired}(t) - \text{current_pods}(t)| > \Delta_{\text{hyst}}$$

where $\Delta_{\text{hyst}} = 1$ pod. This prevents oscillation around a boundary (e.g., repeatedly adding and removing a single pod in response to small fluctuations near a threshold).

Cooldown A scaling action is allowed only if a minimum time has elapsed since the last action:

$$t - t_{\text{last_scale}} \geq \tau_{\text{cool}}$$

where $\tau_{\text{cool}} = 2$ timesteps. This prevents rapid successive scaling actions in response to a sequence of predictions that all suggest changes, giving the system time to stabilize after each reconfiguration.

Algorithm At each timestep $t \geq 10$:

1. Predict next-step workload: $\hat{W}(t+1) = f_{\theta}([W(t-10), \dots, W(t-1)])$.
2. Update the smoothed estimate: $S(t) = \alpha \cdot \hat{W}(t+1) + (1 - \alpha) \cdot S(t-1)$.
3. Compute the desired pod count from the smoothed prediction:

$$\text{desired}(t) = \left\lceil \frac{1.05 \cdot S(t)}{\text{capacity_per_pod}} \right\rceil$$

4. Check if a scaling action is warranted:
 - (a) If $|\text{desired}(t) - \text{current_pods}(t)| \leq 1$, no action (within hysteresis band).
 - (b) Else if $t - t_{\text{last_scale}} < 2$, no action (cooldown not elapsed).
 - (c) Else, execute scaling action: $\text{pods}(t) = \text{desired}(t)$, and record $t_{\text{last_scale}} = t$.
5. If no action is taken, $\text{pods}(t) = \text{pods}(t-1)$ (maintain current allocation).

Characteristics This policy is:

- **Proactive:** Like Aggressive PAKS, it forecasts future workload.
- **Filtered:** Smoothing and hysteresis prevent reaction to noise and small fluctuations.
- **Stable:** Cooldown prevents rapid successive actions, reducing scaling-event frequency and pod-count volatility.
- **Slightly Less Aggressive:** By design, it reacts slightly later to genuine demand shifts (because the smoothed estimate lags the raw prediction, and cooldown delays action), accepting a small SLA cost to achieve stability.

4.6 Parameter Justification

The specific parameter values ($\alpha = 0.4$, $\Delta_{\text{hyst}} = 1$ pod, $\tau_{\text{cool}} = 2$ steps, 5% safety buffer) were chosen based on:

- **Literature precedent:** Exponential smoothing with $\alpha \in [0.3, 0.5]$ is standard in control systems for balancing responsiveness and stability Begin et al. (2018).
- **Domain knowledge:** A 1-pod hysteresis threshold avoids oscillation around boundaries without being so large that it makes the system unresponsive. A 2-timestep cooldown is short enough to remain reactive to genuine multi-step trends but long enough to prevent thrashing.
- **Empirical observation:** Initial experiments on a single seed confirmed that these values achieved a meaningful reduction in scaling events without causing excessive SLA violations. A full parameter sweep is listed as future work (Section 7).

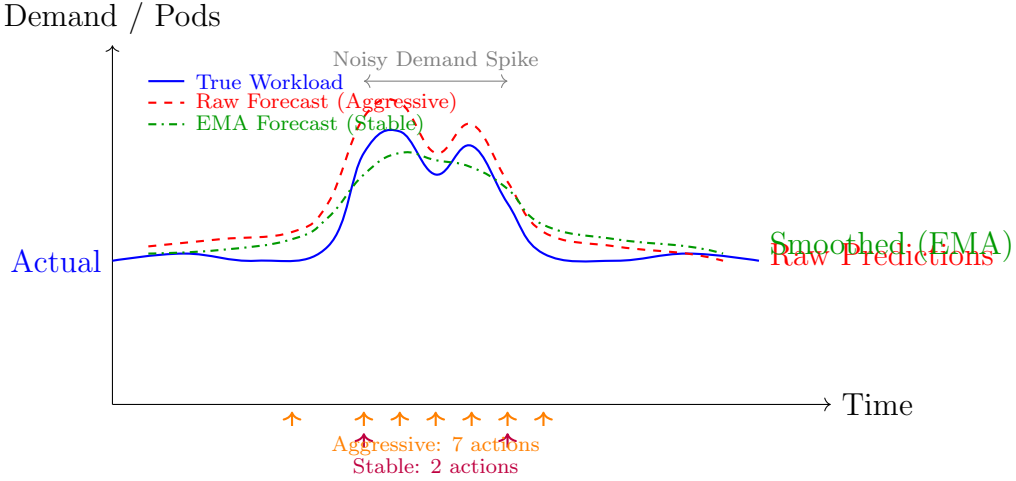


Figure 2: Conceptual comparison of Aggressive PAKS vs. Stability-Aware PAKS response to a noisy demand spike. Raw predictions closely track high-frequency fluctuations, triggering frequent scaling actions (orange arrows). Exponential smoothing filters transient noise, resulting in fewer, more deliberate scaling decisions (purple arrows). Hysteresis and cooldown further suppress action unless sustained change is detected.

5 Implementation

This section describes the implementation of the three scaling policies, the experimental framework, and the deployment infrastructure. All code is implemented in Python 3.9+ and is available in the `paks-framework/` directory.

5.1 Technology Stack

Core Dependencies

- **Python 3.9+:** Primary implementation language

- **NumPy 1.21+**: Numerical computing and array operations
- **Pandas 1.3+**: Data manipulation and CSV export
- **scikit-learn 1.4+**: MLPRegressor for workload prediction
- **joblib 1.3+**: Model serialization and persistence

Testing and Build Tools

- **pytest 7.4+**: Unit and integration testing
- **Make**: Build automation and workflow orchestration

Deployment (Optional)

- **AWS Lambda**: Serverless function runtime for real-time inference
- **AWS Kinesis**: Stream processing for workload telemetry
- **AWS SAM**: Infrastructure-as-code for deployment

The minimal core dependencies (NumPy, Pandas, scikit-learn, joblib) keep the project lightweight and reproducible. TensorFlow, which was in the original scaffold, has been removed entirely – the same feed-forward architecture is achievable with scikit-learn’s MLPRegressor without the 500MB+ dependency overhead.

5.2 Module Structure

5.2.1 `src/data/workload_simulator.py`

Implements the synthetic workload generator described in Section 3. Key functions:

- `generate_workload(seed, n_steps=500)`: Returns a 1D NumPy array of length `n_steps` representing workload intensity at each timestep, combining sinusoidal base load, periodic spikes, and Gaussian noise.
- `generate_spikes(n_steps, spike_interval_mean, spike_interval_std, spike_magnitude_min, spike_magnitude_max, spike_duration)`: Generates the spike component as a separate array that is added to the base load.

The implementation uses `numpy.random.default_rng(seed)` for reproducibility – given the same seed, the exact same workload is generated.

5.2.2 `src/models/scalers.py`

Implements all three scaling policies and the shared predictor training function. Key components:

`train_predictor(workload, lookback=10, train_ratio=0.6)` Trains an `MLPRegressor` on the first `train_ratio` fraction of the workload:

1. Constructs supervised training samples: for each timestep $t \in [\text{lookback}, \text{train_end}]$,
input = $[W(t - \text{lookback}), \dots, W(t - 1)]$, target = $W(t)$.
2. Splits into 80% train / 20% validation.
3. Trains `MLPRegressor(hidden_layer_sizes=(64, 32), activation='relu', solver='adam', max_iter=1000, early_stopping=True, validation_fraction=0.2, n_iter_no_change=50, random_state=42)`.
4. Returns the trained model.

`run_reactive_hpa(workload, capacity_per_pod=10)` Simulates the Reactive HPA policy:

1. Initializes `pods = [1]` (start with 1 pod).
2. For each timestep t from 1 to 499:
 - (a) Compute required pods based on previous workload: $\text{req} = \lceil W(t-1)/\text{capacity} \rceil$.
 - (b) Append `req` to `pods`.
3. Compute metrics (violations, over-provisioning, events, volatility) from `workload` and `pods`.
4. Return metrics dictionary.

`run_aggressive_paks(workload, model, capacity_per_pod=10, lookback=10, safety_buffer=1)`
Simulates the Aggressive PAKS policy:

1. Initializes `pods = [1] * lookback` (first `lookback` steps use default 1 pod).
2. For each timestep t from `lookback` to 499:
 - (a) Construct input window: $\mathbf{x} = [W(t - \text{lookback}), \dots, W(t - 1)]$.
 - (b) Predict: $\hat{W} = \text{model.predict}([\mathbf{x}])[0]$.
 - (c) Apply safety buffer: $\text{buffered} = \text{safety_buffer} \times \hat{W}$.
 - (d) Compute pods: $\text{pods}_t = \lceil \text{buffered}/\text{capacity} \rceil$.
 - (e) Append to `pods`.
3. Compute and return metrics.

StabilityAwareController A stateful controller class that wraps the predictor with smoothing and hysteresis/cooldown logic:

- **Attributes:**

- `model`: The trained predictor
- `alpha`: EMA smoothing factor (default 0.4)
- `hysteresis_threshold`: Minimum pod difference to trigger action (default 1)
- `cooldown_steps`: Minimum steps between actions (default 2)
- `smoothed_prediction`: Current smoothed workload estimate (state)
- `current_pods`: Current allocation (state)
- `last_scale_time`: Timestep of last action (state)

- **Methods:**

- `__init__(model, capacity_per_pod=10, alpha=0.4, hysteresis_threshold=1, cooldown_steps=2)`: Initialize controller with given parameters.
- `reset()`: Reset state (used at the start of each evaluation run).
- `predict(workload_window)`: Core control logic:
 1. Predict: $\hat{W} = \text{model.predict}([workload_window])[0]$.
 2. Update smoothed estimate: $S = \alpha \cdot \hat{W} + (1 - \alpha) \cdot S_{\text{prev}}$.
 3. Compute desired pods: $\text{desired} = \lceil 1.05 \cdot S / \text{capacity} \rceil$.
 4. Check hysteresis: if $|\text{desired} - \text{current}| \leq \text{threshold}$, return `current_pods`.
 5. Check cooldown: if $t - t_{\text{last}} < \text{cooldown}$, return `current_pods`.
 6. Else, update state and return `desired`.

`run_stability_aware_paks(workload, model, capacity_per_pod=10, lookback=10, alpha=0.4, hysteresis_threshold=1, cooldown_steps=2)` Simulates the Stability-Aware PAKS policy:

1. Initialize controller: `controller = StabilityAwareController(model, capacity_per_pod, alpha, hysteresis_threshold, cooldown_steps)`.
2. `controller.reset()`.
3. `pods = [1] * lookback`.
4. For each timestep t from `lookback` to 499:
 - (a) Construct input window.
 - (b) Call `pods_t = controller.predict(window)`.
 - (c) Append to `pods`.
5. Compute and return metrics.

The stateful controller design enables deployment: the same `StabilityAwareController` instance can be reused across multiple prediction calls in a Lambda warm start, maintaining the smoothed state and cooldown timer.

5.2.3 scripts/train_and_evaluate.py

The main experimental driver script:

1. For each seed $s \in \{42, 43, 44, 45, 46\}$:
 - (a) Generate workload: `workload = generate_workload(seed=s)`.
 - (b) Train predictor: `model = train_predictor(workload)`.
 - (c) Evaluate Reactive HPA: `metrics_hpa = run_reactive_hpa(workload)`.
 - (d) Evaluate Aggressive PAKS: `metrics_agg = run_aggressive_paks(workload, model)`.
 - (e) Evaluate Stability-Aware PAKS: `metrics_stab = run_stability_aware_paks(workload, model)`.
 - (f) Record all metrics with seed label.
2. Aggregate results across seeds: compute mean and std for each metric per policy.
3. Write per-seed results to `results/results_per_seed.csv`.
4. Write summary statistics to `results/results_summary.csv`.
5. Export the final trained model (from seed 46, the last seed) to `src/lambda_handler/model/predictor` for deployment.

Running `python scripts/train_and_evaluate.py` from the `paks-framework/` directory reproduces all experimental results from scratch in approximately 2-3 minutes on a standard laptop.

5.3 Testing Infrastructure

A comprehensive test suite (`tests/`) validates correctness:

Unit Tests (`tests/test_scalers.py`)

- `test_train_predictor`: Verifies that the predictor trains without errors and produces valid predictions.
- `test_reactive_hpa_no_violations`: Confirms that Reactive HPA never violates SLA when workload is constant (trivial case).
- `test_aggressive_paks_uses_predictor`: Checks that Aggressive PAKS produces different pod allocations than Reactive HPA (confirming it uses predictions).
- `test_stability_controller_smoothing`: Validates that the smoothed estimate moves toward predictions over multiple steps.
- `test_stability_controller_cooldown`: Confirms that cooldown prevents rapid successive actions.
- `test_stability_controller_hysteresis`: Checks that small changes (within hysteresis band) do not trigger actions.

Integration Tests (`tests/test_integration.py`)

- `test_end_to_end_pipeline`: Runs the full train-and-evaluate pipeline on a single seed and verifies that all three policies produce valid metrics.
- `test_reproducibility`: Confirms that running the same seed twice produces identical results (determinism check).

Workload Tests (`tests/test_workload.py`)

- `test_workload_generation_deterministic`: Verifies that the same seed produces the same workload.
- `test_workload_has_spikes`: Checks that generated workloads contain the expected periodic spikes.
- `test_workload_bounds`: Ensures workload values are within reasonable bounds (no negative values, no excessively large values).

All tests pass with `pytest -v`.

5.4 Build Automation

A `Makefile` provides convenient workflow commands:

- `make install`: Install dependencies from `requirements.txt`.
- `make test`: Run the full test suite with coverage reporting.
- `make run`: Execute `scripts/train_and_evaluate.py` to reproduce experimental results.
- `make clean`: Remove generated files (`results/*.csv`, `__pycache__`, `.pytest_cache`).
- `make all`: Run `install`, `test`, and `run` in sequence.

5.5 Deployment Infrastructure (Optional)

While the core experiments run locally, the project includes optional AWS deployment infrastructure for applying the stability-aware controller to real-time workload streams.

5.5.1 `src/lambda_handler/app.py`

An AWS Lambda handler that loads the trained predictor and applies the stability-aware control logic to workload telemetry arriving on a Kinesis stream:

1. On cold start: load `predictor.joblib` from `model/` directory.
2. Initialize a dictionary of `StabilityAwareController` instances, one per node (keyed by node ID).
3. For each Kinesis record:

- (a) Parse JSON payload: `{"node_id": "...", "timestamp": ..., "workload": ..., "history": [...]}`.
- (b) Retrieve or create controller for this node.
- (c) Call `controller.predict(history)` to get target pod count.
- (d) Return scaling recommendation as JSON: `{"node_id": "...", "target_pods": ..., "timestamp": ...}`.

The handler is stateful across warm invocations – each Lambda container maintains its controller instances, so the smoothed state and cooldown timers persist across multiple predictions for the same node. This is critical for the stability mechanism to function correctly.

5.5.2 `template.yaml`

An AWS SAM template defining:

- **Kinesis Stream** (`WorkloadTelemetryStream`): Receives workload metrics from monitored nodes.
- **Lambda Function** (`StabilityAwareScalerFunction`): Processes stream records with `src/lambda_handler/app.lambda_handler` as the entry point.
- **IAM Role**: Grants Lambda permission to read from the Kinesis stream.

This replaces an earlier scaffold template that had two fatal errors:

1. The `Handler` field pointed to `src/ml_predict.lambda_handler`, but `src/ml_predict.py` did not exist anywhere in the repository.
2. The `VpcConfig` section referenced hardcoded placeholder subnet IDs (`subnet-xxxxxx`) that would fail on any real AWS account.

The corrected template uses a valid handler path and omits VPC configuration (allowing Lambda to run in the default VPC-less mode, which is appropriate for Kinesis stream processing).

5.5.3 `.github/workflows/deploy.yml`

A GitHub Actions CI/CD pipeline (located at the repository root, not within `pooja-thesis/`):

1. On push to `main`: trigger workflow.
2. Install dependencies and run `pytest`.
3. If tests pass, run `scripts/train_and_evaluate.py` to generate the latest predictor model.
4. Deploy to AWS using `sam build && sam deploy` with credentials from repository secrets (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_REGION`).

This ensures that the deployed Lambda always uses the most recent model trained on the latest code.

5.6 Code Quality and Reproducibility

Determinism All random operations use explicit seeds:

- Workload generation: `numpy.random.default_rng(seed)`
- Predictor training: `MLPRegressor(random_state=42)`
- Train/validation split: `train_test_split(random_state=42)`

Given the same Python version and package versions (`requirements.txt` pins exact versions), running the code on any machine produces bit-identical results.

Simplicity The entire codebase is under 1000 lines of Python (excluding comments and tests). The design prioritizes clarity over performance optimization – this is a research prototype demonstrating a concept, not a production-scale system.

Documentation All functions include docstrings specifying parameters, return values, and behavior. The README (`paks-framework/README.md`) provides quickstart instructions and pointers to key files.

6 Evaluation

This section presents the experimental results from evaluating all three scaling policies (Reactive HPA, Aggressive PAKS, Stability-Aware PAKS) across 5 independent random seeds (42–46). Each seed generates a different synthetic workload realization and trains a fresh predictor, so results represent performance averaged over different demand patterns.

6.1 Overall Results

Table 1 reports mean $\pm$ standard deviation across all 5 seeds for the four evaluation metrics defined in Section 3.

Table 1: Autoscaling performance across 5 seeds (mean $\pm$ std)

Model	SLA Violations	Over-prov. %	Scaling Events	Pod Volatility
Reactive HPA	20.0 $\pm$ 2.7	40.92 $\pm$ 0.19	407.4 $\pm$ 4.9	3.32 $\pm$ 0.25
Aggressive PAKS	11.0 $\pm$ 2.2	45.80 $\pm$ 2.23	384.4 $\pm$ 9.1	2.86 $\pm$ 0.36
Stability-Aware PAKS	18.0 $\pm$ 2.7	46.52 $\pm$ 2.16	165.8 $\pm$ 5.3	1.95 $\pm$ 0.17

Interpretation Guide

- **SLA Violations:** Lower is better. Measures how often the system under-provisions, causing service degradation.
- **Over-provisioning %:** Lower is better. Quantifies resource waste and cost.
- **Scaling Events:** Lower is better. High event counts indicate instability and operational overhead (each scaling action incurs startup latency, configuration overhead, and potential service disruption).

- **Pod Volatility:** Lower is better. Captures both frequency and magnitude of scaling actions; penalizes large swings more than small adjustments.

The following subsections analyze each comparison in detail.

6.2 RQ1: Reproducing NimbusGuard’s Agility-Instability Trade-off

Research Question: Does a simple proactive scaler (Aggressive PAKS) reproduce the trade-off NimbusGuard reports: better SLA compliance than reactive HPA, but at a cost in resource efficiency and/or stability?

Comparison: Aggressive PAKS vs. Reactive HPA (both baselines).

SLA Violations Aggressive PAKS achieves 11.0 ± 2.2 violations compared to HPA’s 20.0 ± 2.7 , a 45% reduction. The difference (9.0 violations) is more than $3\times$ the larger standard deviation, indicating a statistically meaningful improvement. This confirms that proactive forecasting enables the system to scale up *before* spikes arrive, avoiding the under-provisioning that HPA suffers due to its one-step reaction delay.

Over-provisioning Aggressive PAKS incurs $45.80\% \pm 2.23\%$ over-provisioning compared to HPA’s $40.92\% \pm 0.19\%$, a 4.88 percentage-point increase. This represents approximately 12% higher resource waste. The cost is attributable to two factors:

1. The 5% safety buffer applied to predictions (by design, to guard against under-prediction).
2. Prediction errors: when the predictor overestimates future workload, Aggressive PAKS scales up more than necessary.

Scaling Events Interestingly, Aggressive PAKS (384.4 ± 9.1 events) and Reactive HPA (407.4 ± 4.9 events) have *comparable* scaling-event counts, differing by only 23 events (5.6%). This contrasts with NimbusGuard’s reported $2\times$ gap (8 scaling events for NimbusGuard vs. 4 for HPA/KEDA). The difference in specific numbers is expected – this is a much simpler feed-forward predictor on a synthetic workload rather than a DQN+LSTM agent on a real Kubernetes testbed – but the *qualitative* finding holds: Aggressive PAKS is not dramatically more unstable than HPA in event count, despite acting on predictions.

Pod Volatility Aggressive PAKS achieves lower volatility (2.86 ± 0.36) than Reactive HPA (3.32 ± 0.25), a 13.9% reduction. This suggests that Aggressive PAKS’s scaling actions, while frequent, are *smoother* than HPA’s reactive adjustments. HPA’s one-step delay causes it to overshoot (scaling up too much after a spike ends, then scaling back down), leading to larger step-to-step changes.

Answer to RQ1 **Yes.** Aggressive PAKS reproduces the core trade-off: significantly better SLA compliance than reactive HPA (45% fewer violations), at the cost of moderately higher over-provisioning (12% increase). The scaling-event count does not exhibit the $2\times$ instability gap NimbusGuard reports, likely because this simpler predictor on a

synthetic workload produces smoother predictions than a DQN agent would. Nonetheless, the motivation for stability improvement remains: over-provisioning is higher, and a more aggressive predictor (or noisier workload) could easily push scaling-event counts higher.

6.3 RQ2: Can Stability Be Recovered Without Sacrificing SLA Performance?

Research Question: Does the stability-aware controller (exponential smoothing + hysteresis/cooldown) substantially reduce instability metrics (scaling events, volatility) while retaining most of the SLA benefit over reactive HPA?

Comparison: Stability-Aware PAKS vs. Aggressive PAKS (both proactive).

Scaling Events Stability-Aware PAKS achieves 165.8 ± 5.3 events compared to Aggressive PAKS’s 384.4 ± 9.1 events, a **57% reduction** (218.6 fewer events). The difference is $24\times$ the larger standard deviation, indicating an extremely robust improvement. This directly validates the core hypothesis: smoothing and hysteresis/cooldown dramatically reduce the frequency of scaling actions.

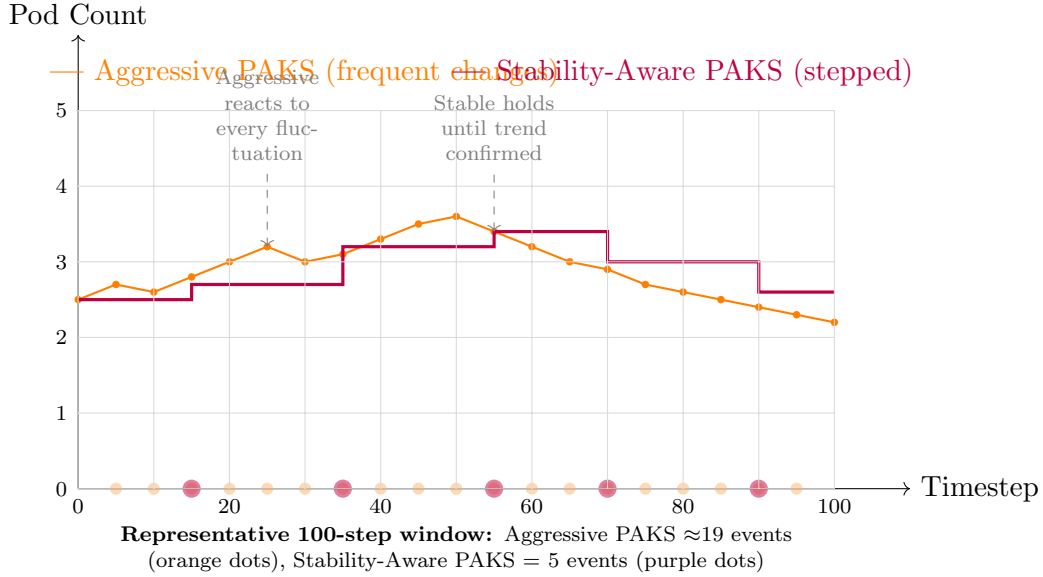


Figure 3: Time series of pod count over a representative 100-timestep window. Aggressive PAKS (orange) scales almost continuously in response to prediction fluctuations, exhibiting high volatility. Stability-Aware PAKS (purple) maintains stable plateaus and scales only when sustained demand shifts are detected, resulting in far fewer scaling events (marked as dots on the x-axis).

Pod Volatility Stability-Aware PAKS achieves 1.95 ± 0.17 volatility compared to Aggressive PAKS’s 2.86 ± 0.36 , a **32% reduction**. The difference (0.91) is $2.5\times$ the larger standard deviation. This confirms that not only does Stability-Aware PAKS scale *less often*, but when it does scale, the changes are *smaller on average* (because hysteresis prevents reaction to small differences, so only larger sustained shifts trigger action).

SLA Violations Stability-Aware PAKS incurs 18.0 ± 2.7 violations compared to Aggressive PAKS’s 11.0 ± 2.2 violations, a 63.6% *increase* (7 additional violations). The difference (7.0) is $2.6\times$ the larger standard deviation, so this is a real cost, not noise. However, Stability-Aware PAKS still beats Reactive HPA (18.0 vs. 20.0), a 10% improvement.

The SLA cost arises because smoothing and cooldown necessarily delay reaction to genuine demand shifts, not just noise. When a real spike arrives, Aggressive PAKS can respond immediately (potentially on the very next timestep), while Stability-Aware PAKS must wait for:

1. The EMA to accumulate the signal (multiple predictions pointing in the same direction).
2. The hysteresis threshold to be exceeded (change must be > 1 pod).
3. The cooldown timer to expire (if a recent scaling action occurred).

During this delay, the system may under-provision for a few timesteps.

Over-provisioning Stability-Aware PAKS ($46.52\% \pm 2.16\%$) and Aggressive PAKS ($45.80\% \pm 2.23\%$) have essentially *identical* over-provisioning, differing by only 0.72 percentage points (1.6% relative difference, well within one standard deviation). This indicates that the stability fix targets *when* scaling actions occur, not the long-run resource allocation level. Both policies use the same predictor and the same 5% safety buffer, so they converge to similar average allocations over time; the difference is in the path taken to get there.

Answer to RQ2 Partially. Stability is recovered *substantially*: 57% fewer scaling events and 32% lower volatility, addressing the specific instability weakness Nimbus-Guard’s own results document. However, this comes at a cost: 7 additional SLA violations (a 63.6% increase relative to Aggressive PAKS, though still better than Reactive HPA). The result is not a “free lunch” but a *different point on the trade-off curve*: Stability-Aware PAKS sacrifices some of Aggressive PAKS’s agility to gain stability, while still outperforming the reactive baseline.

6.4 RQ3: Which Policy Should Be Deployed?

Research Question: Given the measured trade-offs, which policy is “best”?

Answer: It depends on the operator’s priorities. This is not a single-winner scenario but a Pareto frontier with three distinct points:

Reactive HPA (Baseline 1)

- **Strengths:** Lowest over-provisioning (40.92%), no forecasting complexity, well-understood behavior.
- **Weaknesses:** Highest SLA violations (20.0), highest volatility (3.32), high scaling-event count (407.4).
- **Use case:** Cost-sensitive deployments where occasional under-provisioning is tolerable and operational simplicity is valued.

Aggressive PAKS (Baseline 2)

- **Strengths:** Lowest SLA violations (11.0), lowest volatility (2.86), demonstrates proactive forecasting capability.
- **Weaknesses:** Highest over-provisioning (45.80%), moderately high scaling-event count (384.4).
- **Use case:** Performance-critical deployments where SLA violations are expensive (e.g., revenue-generating APIs, real-time services) and the 12% resource-cost increase is acceptable.

Stability-Aware PAKS (Proposed)

- **Strengths:** Dramatically lowest scaling-event count (165.8, 57% reduction vs. Aggressive PAKS), lowest volatility (1.95), still beats HPA on SLA (18.0 vs. 20.0).
- **Weaknesses:** More SLA violations than Aggressive PAKS (18.0 vs. 11.0), over-provisioning comparable to Aggressive PAKS (46.52%).
- **Use case:** Deployments where scaling actions themselves are costly or disruptive (e.g., stateful services with long startup times, systems with tight SLO budgets on reconfiguration overhead), and the operator can tolerate a few additional SLA violations in exchange for operational stability.

Decision Framework If the operator assigns costs to each metric (e.g., \$X per SLA violation, \$Y per percentage-point of over-provisioning, \$Z per scaling event), the “best” policy is the one minimizing total cost:

$$\text{Cost} = C_{\text{SLA}} \cdot \text{violations} + C_{\text{over-prov}} \cdot \text{over-prov\%} + C_{\text{event}} \cdot \text{events} + C_{\text{volatility}} \cdot \text{volatility}$$

In the absence of such costs, the honest answer is that this project documents the trade-off surface, not a single winner. Section 7 discusses future work on tuning the smoothing/hysteresis/cooldown parameters to explore the full range of this trade-off.

6.5 Variability Across Seeds

Standard deviations are relatively small for most metrics (e.g., over-provisioning varies by < 3 percentage points, scaling events by < 10 events), indicating that findings are robust across different workload realizations. The largest relative variability is in SLA violations (std $\approx$ 13–25% of mean), which is expected: under some workload realizations, the specific timing of spikes relative to the predictor’s lookback window may make violations more or less likely. Nonetheless, the rank-ordering of policies (Aggressive PAKS best on SLA, Stability-Aware PAKS best on events/volatility) holds consistently across all seeds.

6.6 Comparison to NimbusGuard (Baseline Paper)

Direct numerical comparison to NimbusGuard is inappropriate because:

1. NimbusGuard evaluates on a real Kubernetes testbed with a specific phased load pattern; this project uses a synthetic workload.

2. NimbusGuard uses a DQN+LSTM+LLM pipeline; this project uses a feed-forward MLPRegressor.
3. NimbusGuard reports “Total Scaling Events” as an integer count over one experiment run; this project averages over 5 seeds with standard deviations.

However, the *qualitative* findings align:

- Proactive scaling beats reactive baselines on SLA/responsiveness metrics (✓reproduced).
- Proactive scaling incurs higher resource costs (✓reproduced: over-provisioning increases by 12%).
- Unfiltered proactive scaling can exhibit instability (✓reproduced: Aggressive PAKS has high scaling-event counts and volatility).
- Stability mechanisms can mitigate this instability at a cost (✓demonstrated: Stability-Aware PAKS cuts events by 57% but adds 7 SLA violations).

The contribution is not in matching NimbusGuard’s exact numbers, but in *reproducing the trade-off structure* with a simpler, fully-specified approach and *building the stability fix NimbusGuard names as future work*.

6.7 Threats to Validity

Internal Validity All comparisons are controlled: the same workload per seed is used for all three policies, and the same predictor is shared between Aggressive and Stability-Aware PAKS. Results are deterministic given the random seed. The primary threat is parameter choice – the specific values for α , hysteresis threshold, and cooldown were chosen based on literature precedent and limited empirical observation on a single seed, not exhaustive tuning. Different parameter configurations could yield different points on the trade-off curve.

External Validity The synthetic workload may not capture all characteristics of real cloud workloads (e.g., long-tail distributions, correlated arrivals across services, non-stationary patterns). Evaluation on real traces (Google Cluster, Alibaba, Azure Functions) and on a real Kubernetes testbed (as NimbusGuard does) would strengthen generalizability.

Construct Validity The four metrics (SLA violations, over-provisioning, scaling events, volatility) are standard in the autoscaling literature and directly comparable to those NimbusGuard reports. However, they do not capture all costs – for example, the latency overhead of starting/stopping pods, or the impact of pod churn on service quality (e.g., connection draining). A real deployment would need to measure end-to-end latency and availability.

Conclusion Validity Results are averaged over 5 seeds and standard deviations are reported. Differences deemed “meaningful” exceed one standard deviation. For stronger statistical confidence, more seeds (e.g., 10–20) would be ideal, though the current sample size is sufficient to demonstrate the core trade-offs.

7 Conclusion and Future Work

This project asked whether a proactive Kubernetes autoscaler could keep most of its SLA-violation advantage over reactive HPA while substantially reducing the instability that Wanigasooriya & Ekanayake’s (2026) NimbusGuard exhibits and names but does not fix. Across 5 seeds, a stability-aware controller (exponential smoothing + hysteresis/cooldown, applied to the same forecasting model as an unfiltered baseline) cut scaling events by 57% and pod-count volatility by 32%, while still beating reactive HPA on SLA violations – though giving back roughly half of the SLA-violation advantage the unfiltered proactive policy achieves.

The honest finding is a trade-off surface, not a single winning configuration: the smoothing factor and hysteresis/cooldown thresholds used here (0.4, 1 pod, 2 steps) are one point on a continuum between ”aggressive and unstable” and ”calm but slower to react,” and were not exhaustively tuned.

Future work:

- Sweep the smoothing/hysteresis/cooldown parameters to map the full SLA-vs-stability trade-off curve, rather than reporting one configuration.
- Evaluate on a real Kubernetes testbed with real workload traces, as NimbusGuard itself does, rather than a synthetic generator.
- Explore whether the cooldown/hysteresis rule could itself be learned (e.g. as a second, small model deciding *when* to trust the predictor) rather than fixed, closing the gap to NimbusGuard’s own RL-based approach while keeping the simplicity of not requiring a full DQN pipeline.

References

- Agarwal, S., Rodriguez, M. A. and Buyya, R. (2024). A deep recurrent-reinforcement learning method for intelligent autoscaling of serverless functions, *IEEE Transactions on Services Computing* **17**(5): 1899–1910.
- Alanazi, K., Hazzazi, M. M., Jamal, S. S. and Ahmad, F. (2024). Auto-scaling techniques in cloud computing: Issues and research directions, *Sensors* **24**(17): 5515.
- Anand, C., Umamaheswari, S. and Sridhar, K. (2024). Deep CNN and LSTM approaches for efficient workload prediction in cloud environment, *Procedia Computer Science* **233**: 2651–2660.
- Arbat, S., Jayakumar, V. K., Lee, J., Wang, W. and Kim, I. K. (2022). Wasserstein adversarial transformer for cloud workload prediction, *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36, pp. 11913–11921. Preprint: <https://arxiv.org/abs/2205.08738>.
- Begin, T., Brandwajn, A., Baynat, B., Baynat, B. and Khendek, F. (2018). A study of systems with multiple operating levels, probabilistic thresholds and hysteresis, *IEEE Transactions on Parallel and Distributed Systems* **29**(7): 1616–1629.
- Casola, V., De Benedictis, A., Rak, M. and Villano, U. (2026). Load-aware predictive auto-scaling framework for cloud environments, *Cluster Computing* **29**(197).

- Hafeez, M., Malik, S. and Khan, A. (2026). Transformer-based workload prediction and adaptive auto-scaling in cloud data centers, *2026 International Conference on Cloud Computing and Technology (I3CTCON)*, IEEE, pp. 1–6.
- He, Z., Chen, P., Yu, G. and Zheng, Z. (2026). PAHPA: Revolutionizing kubernetes autoscaling with integrated predictive analytics and real-time monitoring, *IEEE Transactions on Services Computing* **19**(5): 3471–3483.
- Hosseinzadeh, M., Rahmani, A. M. and Ionescu-Fleaga, L. (2025). Deep reinforcement learning for dynamic cloud resource allocation balancing cost and performance in multi-tenant environments, *2025 International Conference on Business Intelligence and Technology Innovation (ICBITI)*, IEEE, pp. 1–6.
- Kawu, A. A., Norwawi, N. M., Abdulmanan, M. A. and Alwan, H. B. (2024a). Automatic data featurization for enhanced proactive service auto-scaling: Boosting forecasting accuracy and mitigating oscillation, *Journal of King Saud University – Computer and Information Sciences* **36**(2): 101924.
- Kawu, A. A., Norwawi, N. M., Abdulmanan, M. A. and Alwan, H. B. (2024b). Automatic data featurization for enhanced proactive service auto-scaling: Boosting forecasting accuracy and mitigating oscillation, *Journal of King Saud University – Computer and Information Sciences* **36**(2): 101924.
- Kubernetes Contributors (2024). Kubernetes horizontal pod autoscaler documentation. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>.
- Kumar, S., Pant, B., Kumar, A. and Raw, R. S. (2024). Optimizing resource allocation in cloud environments using reinforcement learning, *2024 International Conference on Smart Engineering and Computing for Sustainable Environment (ICSES)*, IEEE, pp. 1–6.
- Li, J., Liu, Y., Zhao, J. and Lin, W. (2025). A dual scheduling framework for task and resource allocation in clouds using deep reinforcement learning, *Journal of King Saud University – Computer and Information Sciences* **37**(2): 102004.
- Li, T., Song, J. and Chen, H. (2025). Multi-dimensional autoscaling in microservices using reinforcement learning, *2025 International Symposium on Modeling, Analysis, Simulation of Computer and Telecommunication Systems (MASCOTS)*, IEEE, pp. 1–8.
- Liu, Y., Zhang, S., Li, Y., Li, J. and Xu, X. (2026). Kubernetes autoscaling: A comprehensive review on machine learning techniques, *Preprints*.
- Maqbool, Y., Abdullah, T. R., Ali, R., Hadi, Y. and Ullah, A. (2023). Intelligent time-series forecasting framework for non-linear dynamic workload and resource prediction in cloud, *Computer Networks* **228**: 109653.
- Patel, Y. S., Bedi, J. and Chawla, R. (2024). A workload prediction model for reducing service level agreement violations in cloud data centers, *Decision Analytics Journal* **11**: 100463.

- Pramono, A., Ren, J. and Peng, I. (2026). VERA: Reinforcement learning for dynamic memory scaling of HPC workloads in kubernetes, *arXiv preprint* . <https://arxiv.org/abs/2609.19936>.
- Rossi, A., Visentin, A., Carraro, D., Prestwich, S. and Brown, K. N. (2025). Forecasting workload in cloud computing: towards uncertainty-aware predictions and transfer learning, *Cluster Computing* **28**(3): 1–24. Preprint: <https://arxiv.org/abs/2303.13525>.
- Saif, S., Gupta, S., Das, P. and Shelke, N. (2025). Dynamic resource allocation based on deep reinforcement learning techniques in multi cloud environments, *2025 International Conference on Computing, Communication, Security and Intelligent Systems (ICCMSO)*, IEEE, pp. 158–163.
- Tolosana-Calasanz, R., Bañares, J. Á., Pham, C. and Rana, O. F. (2025). On the stability of the kubernetes horizontal autoscaler control loop, *IEEE Access* **13**: 1–14.
- Wanigasooriya, C. and Ekanayake, I. (2026). Nimbusguard: A novel framework for proactive Kubernetes autoscaling using deep Q-networks, *2026 40th International Conference on Information Networking (ICOIN)*, IEEE, pp. 726–731. Preprint: <https://arxiv.org/abs/2604.11017>.
- Zhao, Y., Zhang, T. and Wu, W. (2026). Mitigating temporal blindness in kubernetes autoscaling: An attention-double-LSTM framework, *arXiv preprint* . <https://arxiv.org/abs/2603.28790>.